Sprint 4 · Reto «Herramientas internas»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad el reto en una página.

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 Qué os pide el cliente

Qué os pide el cliente. El equipo de reservas de TurisData calcula cada semana sus cifras **a mano en una hoja de cálculo**: es lento y cada persona lo hace distinto. Os pide una **herramienta interna**, un programa en Python que lea el fichero de reservas, calcule **tres KPI** (cifras que resumen cómo va el negocio: la ocupación, el ingreso medio por reserva y la tasa de cancelación por canal), **avise con claridad si el fichero está mal** y guarde el resultado en un fichero JSON que otros sistemas puedan leer.

Cómo se trabaja. No hace falta experiencia previa: os damos una **plantilla casi hecha** (funciones vacías y pruebas que os dicen si lo que escribís es correcto) y vuestro trabajo es rellenarla paso a paso. La ocupación será solo un *proxy* (una medida aproximada), porque el fichero no trae la capacidad real del hotel. En clase tenéis 10 minutos de planning (día 1) y **100 minutos de reto + 15 de entrega** (día 2); lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Python 3.11 (solo biblioteca estándar: `csv`, `json`, `argparse`, `datetime`)	Escribir y ejecutar `kpis.py`	Ya instalado en el aula; si no, el de Colab
Editor y terminal: VS Code (recomendado) o Colab	Editar el código y ejecutar `python test_kpis.py`	En Colab, subid la carpeta y poned `!` delante: `!python test_kpis.py`
La plantilla `reto_herramientas/`	`kpis.py` con funciones vacías, `test_kpis.py`, `README_PLANTILLA.md` y `plantilla_kpis_pandas.py` (extra)	En GitHub: *Code* → *Download ZIP*. Trabajad en una copia
El CSV real `reservas_turisdata.csv`	6000 reservas **sintéticas**	Botón *Download raw file*; dejadlo **junto a `kpis.py`**
Cuadernos del sprint: `S04_03` y `S04_04`	Las pistas para las funciones (diccionarios, CSV, JSON, excepciones)	Colab; su código explicado
Git y GitHub + cuaderno `S04_05`	Guardar versiones (*commits*) y entregar	Terminal o VS Code. Si falla, subid los ficheros desde la web (*Add file* → *Upload files*): cada subida es un commit
Cómo entregar en GitHub	Subir la carpeta `sprint-04/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender errores y pedir pistas	Con las reglas de la «Ayuda 1» del paso a paso

3 · Qué entregar

Qué se sube: `kpis.py`, `test_kpis.py`, `README.md` y `salida.json`, dentro de la carpeta **`sprint-04/`** del repositorio privado del

equipo. Niveles: mínimo (entrega válida) · estándar · extra.

Requerimiento	Qué se pide exactamente	Nivel	Dónde va
Leer el CSV	`leer_reservas`: convierte los tipos y **descarta y cuenta** las filas defectuosas (noches < 1, precio ≤ 0, `cancelada` distinto de 0/1, datos que no se pueden convertir)	Mínimo	`kpis.py`
Tasa de cancelación por canal	canceladas ÷ reservas del canal × 100, con 1 decimal	Mínimo	`kpis.py` · `tasa_cancelacion_por_canal`
Ingreso medio por reserva	media de `noches × precio_noche` de las reservas **no canceladas**, con 2 decimales (`0.0` si no hay)	Mínimo	`kpis.py` · `ingreso_medio_por_reserva`
Ocupación (proxy)	noches no canceladas ÷ (días del periodo × habitaciones) × 100; el periodo va de la primera a la última llegada. En el mínimo, con ayuda del docente	Estándar	`kpis.py` · `ocupacion_proxy`
Guardar el resultado	`exportar_json` y `main`: JSON en UTF-8 con los tres KPI y la sección `calidad` (filas válidas, descartadas y avisos). En el mínimo, con ayuda del docente	Estándar	`kpis.py` → `salida.json`
Avisar de los errores	Fichero inexistente o columna que falta → mensaje claro y código de salida 1, sin `Traceback`	Mínimo	`kpis.py` · `main` y `leer_reservas`
Resultado con el CSV real	`python kpis.py reservas_turisdata.csv salida.json --capacidad 40`	Mínimo	`salida.json`
Pruebas	`python test_kpis.py` pasa todo. **No se modifican** las dadas; las vuestras, aparte	Estándar	`test_kpis.py`
Código limpio	Docstring en cada función y estilo PEP 8	Estándar	`kpis.py`
README	Cómo se ejecuta y se prueba · qué es cada KPI · por qué la ocupación es un proxy · uso de la IA (qué pedisteis y qué revisasteis). Parte de `README_PLANTILLA.md`. En el mínimo, versión corta	Mínimo	`README.md`
Git	Al menos 3 *commits* con mensaje claro, de personas distintas, sin claves ni datos personales	Mínimo	Repositorio del equipo, carpeta `sprint-04/`
Revisión y autoevaluación	Alguien del equipo revisa el código con la rúbrica; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	Carpeta `sprint-04/`
Extras	Pruebas propias · `kpis_pandas.py` con los mismos resultados · argumento `--por-mes` · `@dataclass` para `Reserva`	Extra	`test_kpis.py` · `kpis_pandas.py` · `kpis.py`

Cómo se evalúa. El docente puntúa de 0 a 4 con la rúbrica del curso: **corrección técnica** (las pruebas pasan y los KPI coinciden), **reproducibilidad** (otra persona obtiene el mismo JSON con vuestro README), **comunicación al cliente** (README y JSON se entienden sin ser programador) e **IA responsable** (qué se hizo con IA, qué se revisó y los límites de datos sintéticos y proxy). Orientativamente, el mínimo vale un 2, el estándar un 3 y los extras un 4.

---

4 · Guía paso a paso

Cada paso dice cuánto dura, qué se hace y cuándo está hecho. Las ayudas (IA, errores, atascos) están al final.

Paso 1 · Preparar la carpeta y ver que las pruebas fallan (día 2, 10 minutos)

1. Copiad la plantilla y dejad el CSV real junto a `kpis.py` (ver «Recursos del reto»).
2. Abrid una terminal en esa carpeta y ejecutad `python test_kpis.py`.
3. Repartid el trabajo (ver el paso 2).

Hecho cuando: veis las pruebas fallando con `NotImplementedError`. Es lo normal: las funciones están vacías.

Paso 2 · Rellenar las funciones, de una en una (día 2, 45 minutos)

1. **Entre todo el equipo, primero `leer_reservas`** (con `_convertir_fila`): las demás pruebas dependen de ella.
2. Después, repartid el resto: una persona por función, y otra la revisa.

3. Para cada función: programadla, ejecutad `python test_kpis.py`, mirad qué prueba falla, corregid y repetid. Es el método de los profesionales: trabajar **guiados por las pruebas**.
4. Nunca cambiéis `test_kpis.py` para que pase: el código es el que se adapta a las pruebas.

Función	Qué hace	Prueba que la valida	Dónde está la pista
`_convertir_fila` y `leer_reservas`	Lee el CSV, convierte los tipos, **descarta y cuenta** las filas defectuosas	`test_leer_reservas…`, `test_fichero_inexistente`, `test_faltan_columnas`, `test_filas_defectuosas…`, `test_valores_imposibles…`	Cuaderno `S04_04`
`tasa_cancelacion_por_canal`	% de canceladas de cada canal, 1 decimal	`test_tasa_cancelacion…`	Cuaderno `S04_03`
`ingreso_medio_por_reserva`	Media de `noches × precio` de las no canceladas, 2 decimales; `0.0` si no hay	`test_ingreso_medio…`	Cuaderno `S04_03`
`ocupacion_proxy`	La fórmula de la tabla de KPI	`test_ocupacion_proxy`	Docstring de la función

Hecho cuando: esas pruebas pasan. En el nivel mínimo, la ocupación se completa con ayuda del docente.

Paso 3 · Exportar el resultado y ejecutar el programa entero (día 2, 15 minutos)

1. Programad `exportar_json` (UTF-8, `ensure_ascii=False`, sangría de 2) y `main` (leer → calcular → añadir `calidad` → exportar). `calcular_kpis` ya viene hecha.
2. Ejecutad: `python kpis.py reservas_turisdata.csv salida.json --capacidad 40`
3. Abrid `salida.json` y comparadlo con este resultado esperado:

```
{
  "reservas_analizadas": 6000,
  "ocupacion_proxy_pct": 42.0,
  "ingreso_medio_por_reserva": 576.04,
  "tasa_cancelacion_por_canal_pct": {"empresa": 43.3, "directo": 31.5, "web_intermediaria": 53.9, "agencia": 44.0},
  "calidad": {"filas_validas": 6000, "filas_descartadas": 0, "avisos": []}
}
```

Hecho cuando: existe `salida.json` con las mismas cifras y `python test_kpis.py` pasa todo. Si el vuestro difiere, mirad la «Ayuda 2».

Paso 4 · Probar que no se rompe (día 2, 15 minutos)

1. Ejecutad el programa con un fichero que no existe.
2. Quitad una columna al CSV (en una copia) y ejecutad de nuevo.
3. Poned un dato incorrecto en una fila (por ejemplo, un precio con letras).
4. Comprobad en cada caso que el mensaje es claro, sin el texto rojo de `Traceback`:

Caso	Mensaje esperado	Código de salida
1. Fichero que no existe	`ERROR: No se encuentra el fichero: …`	1
2. Falta una columna	`ERROR: Faltan columnas obligatorias: […]`	1
3. Dato incorrecto en una fila	El programa sigue y la fila aparece en `calidad.avisos`	0

Hecho cuando: los tres casos se comportan así. Ese será vuestro «provocar un error» en la review.

Paso 5 · README y Git (día 2, 15 minutos)

1. Copiad `README_PLANTILLA.md` como `README.md` y rellenadlo.
2. Subid el trabajo a `sprint-04/` como indica la tabla de «Qué entregar».
3. Si os da tiempo, empezad el extra: pruebas propias o `plantilla_kpis_pandas.py`.

Mensajes de commit de ejemplo: «Añade lectura del CSV», «Añade KPI y exportación», «Añade README».

Hecho cuando: otra persona podría ejecutar el proyecto siguiendo solo el README.

Después, 15 minutos de entrega y review: subid todo a `sprint-04/`, rellenad la autoevaluación y coevaluación y enseñad, en 3–5 minutos, el script con el CSV real, el JSON, un error provocado con su mensaje y las pruebas en verde. Explicad una decisión de diseño, por ejemplo cómo definisteis la ocupación.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código que parece funcionar y no se entiende**. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.
- **Las pruebas mandan:** si el código de la IA no las pasa, no vale.
- **Declaradlo** en la sección «Uso de IA y límites» del README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un error	«Me sale este error: *[mensaje completo]*. Explícame qué significa, sin darme todavía la solución»	Que lo entendéis y lo arregláis vosotros
Entender una función	«Explícame línea por línea qué hace esta función: *[código]*»	Ejecutadla con un ejemplo pequeño
Pedir una pista	«Quiero calcular el % de cancelaciones por canal con la biblioteca estándar. ¿Qué estructura me conviene? No me des el código»	Que respeta «solo biblioteca estándar»
Revisar el estilo	«¿Sigue PEP 8 este código? ¿Qué cambiarías?»	Que las pruebas siguen pasando

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
`ModuleNotFoundError: kpis`	Se ejecuta `test_kpis.py` desde otra carpeta	Abrir la terminal **dentro** de la carpeta del reto
La media sale rara	Se incluyen reservas canceladas en el ingreso	**Solo las no canceladas**
La ocupación no sale 42.0	Falta el `+ 1` en los días, se cuentan canceladas o la capacidad no es 40	Releer la fórmula y ejecutar con `--capacidad 40`
Las pruebas fallan todas a la vez	Dependen de `leer_reservas`	Empezar por esa función
El programa se rompe con un fichero mal	No se controlan las excepciones	`try` / `except` con tipos concretos y mensajes claros
Tildes rotas	Falta `encoding="utf8"`	Añadirlo al abrir y escribir ficheros
Cambiar las pruebas para que pasen	Prisa	No tocar las pruebas dadas; añadir las propias aparte
Un solo *commit* gigante	«Lo subo al final»	Commits pequeños mientras avanzáis

Ayuda 3 · Si os atascáis

1. Leed el mensaje de la prueba que falla: dice qué esperaba y qué recibió. Probad la función en una celda con un ejemplo pequeño.
2. Repasad el cuaderno que corresponde a esa función (tabla del paso 2).
3. Preguntad al equipo y, después, al docente.